

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

**COURSE NAME:** Artificial Intelligence

**COURSE CODE:** CSA17

---

### COURSE OUTCOME COVERED

**CO2:** Experiment search and game-solving techniques such as Greedy Search, A\*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

*Assessment Tool Weightage: 50%*

---

**QUESTIONS: 5**

**Duration: 1 Hour**  
**Total Marks:50**

Annexure A

### Scenario Based Assignment

---

**Code of Conduct:**

*I **LATISHA S - 192524071** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student:**

## Annexure A

### Scenario Based Assignment

**1. Scenario :** A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A\* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

**2. Scenario:** A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

**Tasks.** Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

**3. Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments

- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

## RUBRICS FOR CALCULATION

| Criteria                     | Weightage (%) | Level 4<br>(Exemplary)                                                    | Level 3<br>(Proficient)                            | Level 2<br>(Developing)                           | Level 1<br>(Beginning)                            |
|------------------------------|---------------|---------------------------------------------------------------------------|----------------------------------------------------|---------------------------------------------------|---------------------------------------------------|
| Understanding of Scenario    | 20%           | Demonstrates clear and complete understanding of the scenario and context | Shows good understanding with minor gaps           | Partial understanding; misses key aspects         | Misinterprets or fails to understand the scenario |
| Identification of Key Issues | 20%           | Accurately identifies all relevant issues and factors involved            | Identifies most key issues with minor omissions    | Identifies limited issues; some irrelevant points | Unable to identify key issues                     |
| Application of Concepts      | 25%           | Applies appropriate concepts effectively to address the scenario          | Applies concepts with minor errors                 | Limited or partially correct application          | Unable to apply relevant concepts                 |
| Decision Making              | 20%           | Makes well-reasoned, logical decisions with strong rationale              | Makes reasonable decisions with some justification | Makes weak decisions with limited justification   | No clear decisions made or rationale provided     |
| Clarity of Response          | 15%           | Response is clear, structured, and logically presented                    | Generally clear with minor issues                  | Somewhat unclear or poorly structured             | Disorganized and difficult to understand          |

# ANSWERS

## 1. AI-Powered Drones for Emergency Medicine Delivery (Greedy Best-First Search vs A\*)

### i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

Greedy Best-First Search (GBFS) selects the path that appears closest to the destination using only the heuristic value  $h(n)$  (straight-line distance). It ignores the actual travel cost.

Working in this scenario

- The drone chooses the route with the shortest estimated distance to the destination.
- It immediately moves toward locations that seem closest, without considering weather conditions or blocked roads.
- When new information about blocked paths is received, it may need to backtrack and search again.

Strengths

- Very fast decision-making.
- Requires less computation.
- Suitable when emergency response is more important than finding the perfect route.
- Uses minimal memory compared to exhaustive search.

Weaknesses

- Does not guarantee the shortest or safest path.
- May repeatedly choose blocked or risky routes.
- Performs poorly when heuristic estimates are inaccurate.
- Frequently replans when unexpected obstacles appear.

### ii. Explain how A\* would handle the same situation, including how it balances cost and heuristic.

A\* evaluates nodes using

$$f(n) = g(n) + h(n)$$

where

- $g(n)$  = actual travel cost from the start.
- $h(n)$  = estimated remaining distance.

Working

The drone evaluates both:

- the distance already travelled,
- the estimated distance remaining.

It selects the path with the lowest total estimated cost.

### Advantages

- Considers terrain difficulty and weather.
- Avoids unnecessarily expensive routes.
- Finds the optimal route if the heuristic is admissible.
- Produces safer navigation decisions.

Example:

Route A:

- Cost travelled = 25
- Estimated remaining = 30
- Total = 55

Route B:

- Cost travelled = 35
- Estimated remaining = 15
- Total = 50

A\* chooses Route B because its total estimated cost is lower.

### iii. Analyze the impact of heuristic accuracy on both algorithms.

The heuristic estimates the remaining distance to the goal.

Accurate heuristic

- GBFS reaches the destination quickly.
- A\* finds the optimal solution faster.

Inaccurate heuristic

- GBFS may follow completely wrong directions.
- A\* still finds the optimal solution but explores more nodes

| Heuristic Accuracy | GBFS              | A*                 |
|--------------------|-------------------|--------------------|
| High               | Very fast         | Fast and optimal   |
| Medium             | May make mistakes | Still reliable     |
| Poor               | Performs badly    | Slower but correct |

Thus, A\* is much less sensitive to heuristic errors.

## 2. Smart City Traffic Signal Optimization (Hill Climbing, Simulated Annealing, Genetic Algorithm)

Problem Formulation

This is an optimization problem.

Objective Function

Minimize:

- Total vehicle waiting time
- Fuel consumption
- Traffic congestion

Decision Variables:

- Green signal duration
- Red signal duration
- Signal coordination between intersections

Why Traditional Search is Inefficient

- Hundreds of intersections create millions of possible signal combinations.
- Traffic conditions change every minute.
- Exhaustive search requires excessive computation.
- Not suitable for real-time optimization.

### **i. Explain how Hill Climbing works and identify its limitations.**

Hill Climbing starts with an initial traffic signal plan.

It:

1. Evaluates the current solution.
2. Makes small adjustments.
3. Accepts changes only if they improve performance.

Example:

Increase green time by 10 seconds if it reduces waiting time.

Advantages

- Simple implementation.
- Fast computation.
- Suitable for local optimization.

Limitations

- Can get stuck in local maxima.
- Cannot escape plateaus.
- Sensitive to starting solution.

### **ii. Discuss local maxima, plateaus, and ridges.**

#### **Local Maximum**

The algorithm reaches a good solution but not the best one.

Example:

One busy junction improves while the overall city traffic remains poor.

#### **Plateau**

Many neighbouring solutions have identical performance.

Example:

Changing signal timings by 2–3 seconds produces no noticeable improvement.

#### **Ridge**

Improvement requires multiple coordinated changes.

Example:

Changing only one intersection has no effect; several intersections must be adjusted together.

**iii. Explain how Simulated Annealing and Genetic Algorithms overcome these limitations.**

**Simulated Annealing**

- Sometimes accepts worse solutions.
- Escapes local maxima.
- Probability of accepting poor solutions decreases over time.

Advantages:

- Avoids premature convergence.
- Suitable for dynamic optimization.

**Genetic Algorithm (GA)**

Inspired by natural evolution.

Steps:

1. Generate population of signal plans.
2. Evaluate fitness.
3. Select best solutions.
4. Perform crossover.
5. Apply mutation.
6. Repeat.

Advantages:

- Searches many solutions simultaneously.
- Escapes local optima.
- Scales well for large problems.

**iv. Recommend the best approach.**

**Recommendation: Genetic Algorithm**

**Justification**

- Handles very large search spaces.
- Optimizes multiple objectives simultaneously.
- Easily adapts to changing traffic.
- Avoids local optima.
- Suitable for smart city traffic management.

**3. Mars Rover Exploration (Online Search Agent)**

**i. Explain why this problem requires an online search agent.**

Offline search assumes:

- Complete map available.
- Entire environment known before movement.

Mars rover:

- Unknown terrain.
- Limited sensors.
- Discovers hazards while moving.

Therefore, an online search agent is required because it learns while exploring.



## **ii. Analyze challenges of partial observability and dynamic environments.**

### **Partial Observability**

The rover can observe only nearby terrain.

Challenges:

- Hidden obstacles.
- Limited sensor range.
- Uncertain paths.

### **Dynamic Environment**

Conditions change.

Examples:

- Dust storms.
- Loose rocks.
- Sand movement.

The rover must continuously update its decisions.

## **iii. Describe how the rover builds and updates its internal model.**

The rover maintains an internal map.

Steps:

1. Sense surroundings.
2. Detect hazards.
3. Update terrain map.
4. Store visited locations.
5. Replan safest route.

The internal model becomes more accurate over time.

## **iv. Compare online search with uninformed and informed search.**

| <b>Feature</b>               | <b>Uninformed Informed Online Search</b> |         |                   |
|------------------------------|------------------------------------------|---------|-------------------|
| Complete map required        | Yes                                      | Yes     | No                |
| Uses heuristic               | No                                       | Yes     | May use heuristic |
| Learns while moving          | No                                       | No      | Yes               |
| Suitable for unknown terrain | No                                       | Limited | Yes               |

## **v. Justify the most suitable approach.**

### **Recommendation: Online Search Agent**

Reasons:

- Works with incomplete information.
- Adapts to new hazards.
- Learns continuously.
- Ensures safer navigation.
- Suitable for real-time autonomous exploration.

#### 4. University Exam Timetable (Constraint Satisfaction Problem)

##### Problem Formulation as CSP

###### Variables:

Each variable represents an examination.

Example:

- Math Exam
- Physics Exam
- Chemistry Exam

###### Domains:

Possible values:

- Time slots
- Exam halls

Example:

Math  $\in$  {Monday Morning, Monday Afternoon, Tuesday Morning}

###### Constraints:

1. No student has overlapping exams.
2. Limited exam halls.
3. Faculty must be available.
4. Subject prerequisites.
5. Special accommodations.
6. One exam per hall at a time.

#### i. Variables, Domains, and Constraints

##### Component Description

Variables    Exams

Domains    Time slots and halls

Constraints   Student, hall, faculty, prerequisite, accommodation constraints

---

#### ii. Explain Backtracking Search.

Backtracking assigns values step by step.

Example:

- Assign Math  $\rightarrow$  Monday Morning
- Assign Physics  $\rightarrow$  Monday Afternoon
- Assign Chemistry  $\rightarrow$  Conflict

Backtrack:

- Change Physics
- Continue search

This continues until all constraints are satisfied.

#### iii. Explain Forward Checking.

Forward Checking removes impossible future assignments after each decision.

Example:

If Hall A is assigned Monday Morning,  
remove Hall A from remaining exams during that slot.

Benefits:

- Detects conflicts early.
- Reduces unnecessary search.
- Speeds up timetable generation.

#### **iv. Real-world challenges and best strategy.**

Challenges:

- Thousands of students.
- Hundreds of courses.
- Faculty availability changes.
- Hall capacity limits.
- Continuous updates.

#### **Best Strategy**

Use:

- Backtracking
- Forward Checking
- Constraint Propagation
- Variable Ordering (MRV heuristic)

This combination greatly improves scalability and efficiency.

### **5. Strategic Game AI (Minimax & Alpha-Beta Pruning)**

#### **i. Explain how Minimax models this problem.**

Minimax assumes two players:

- AI (Max)
- Human (Min)

The AI:

- Builds a game tree.
- Predicts opponent moves.
- Chooses the move that maximizes its minimum guaranteed score.

#### **ii. Analyze computational challenges.**

Large games produce enormous game trees.

Example:

Branching factor = 30

Depth = 8

Nodes evaluated:

$$30^8 = 656,100,000,000$$

Challenges:

- High computation.
- Large memory usage.
- Cannot search the full tree in real time.

### **iii. Explain Alpha-Beta Pruning.**

Alpha-Beta Pruning removes branches that cannot influence the final decision.

**Alpha**

Best score for the maximizing player.

**Beta**

Best score for the minimizing player.

If

$$\alpha \geq \beta$$

remaining branches are pruned.

**Benefits**

- Evaluates far fewer nodes.
- Produces the same optimal move as Minimax.
- Significantly faster.
- Suitable for real-time games.

### **iv. Design an evaluation function.**

A good evaluation function may include:

$$\begin{aligned} \text{Score} = & 5(\text{Resources}) + 10(\text{Army Strength}) + 8(\text{Map Control}) \\ & + 12(\text{Strategic Position}) + 15(\text{Base Safety}) \\ & - 10(\text{Enemy Advantage}) \end{aligned}$$

Factors considered:

- Army size
- Available resources
- Territory controlled
- Base health
- Unit positioning
- Enemy strength

This allows the AI to compare non-terminal game states efficiently.

### **v. Recommend a strategy for balancing optimality and real-time performance.**

**Recommendation**

Use:

- **Minimax with Alpha-Beta Pruning**
- Depth-limited search
- Heuristic evaluation function

- Iterative deepening (if additional time is available)

### **Justification**

- Maintains high-quality decision-making.
- Reduces computation dramatically.
- Meets strict real-time constraints.
- Produces near-optimal moves within the available time.
- Widely used in modern game AI due to its balance of optimality, speed, and scalability.